

Digital Marketing Campaign Conversion Indicator

- **Project Type** : EDA & Machine Learning
- **Contribution** : Individual
- **Created by** : Rishabh Salian

This is a continuation of the EDA section of this Project now let's go towards the Machine learning Model Training & Building.

To Check out the Notebook file for the EDA phase click / copy & paste this link :

https://github.com/Rishabh90501/DMC_Conversion/blob/18763e049e0f45a3addaeaddbd09a6279aab2f214/notebook/DMC_Exploratory



Import Data and Required Packages

```
In [1]: # Base Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

# Import Connectors
from sqlalchemy import create_engine

# Modelling Libraries
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier, IsolationForest
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from catboost import CatBoostClassifier
from xgboost import XGBClassifier
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from imblearn.over_sampling import RandomOverSampler
from sklearn.model_selection import GridSearchCV, cross_val_score, StratifiedKFold
from imblearn.over_sampling import RandomOverSampler
```

```
from sklearn.metrics import classification_report, accuracy_score, precision_score, recall_score, f1_score, log_loss, c
from scipy.stats import uniform, randint
```

Creating Connection engine with MYSQL Database

```
In [2]: # Creating pipeline through sql server
engine = create_engine('mysql+mysqlconnector://root:l4r5a1d7c3@localhost/digital_marketing_campaign')
dm_query = 'select * from dm_campaign'
```

Import & Reading Dataset from SQL Connection as Pandas Dataframe

```
In [3]: # Importing data from the mysql database
df = pd.read_sql(dm_query,engine)
df = df.set_index("CustomerID")

# Backing up the Original data
df_backup = df.copy

# Structuring dataframe and show Top 5 Records
pd.set_option('display.max_columns',None)
pd.set_option('display.width',None)
pd.set_option('display.max_colwidth',None)
df.head(5)
```

```
Out[3]:
```

	Age	Gender	Income	CampaignChannel	CampaignType	AdSpend	ClickThroughRate	ConversionRate	WebsiteVis
CustomerID									
8000	56	Female	136912	Social Media	Awareness	6497.87	0.04	0.09	
8001	69	Male	41760	Email	Retention	3898.67	0.16	0.18	
8002	46	Female	88456	PPC	Awareness	1546.43	0.28	0.08	
8003	32	Female	44085	PPC	Conversion	539.53	0.14	0.09	
8004	60	Female	83964	PPC	Conversion	1678.04	0.25	0.11	

```
In [4]: df2 = df.copy()
df2['total_seconds'] = df2['TimeOnSite'].dt.total_seconds()
df2['minutes'] = (df2['total_seconds'] // 60).astype(int)
df2['seconds'] = (df2['total_seconds'] % 60).astype(int)
df2['time_float'] = df2['minutes'] + df2['seconds'] / 100
df['TimeOnSite'] = df2['time_float']
df.head()
```

Out[4]:

	Age	Gender	Income	CampaignChannel	CampaignType	AdSpend	ClickThroughRate	ConversionRate	WebsiteVis
--	-----	--------	--------	-----------------	--------------	---------	------------------	----------------	------------

CustomerID

8000	56	Female	136912	Social Media	Awareness	6497.87	0.04	0.09
8001	69	Male	41760	Email	Retention	3898.67	0.16	0.18
8002	46	Female	88456	PPC	Awareness	1546.43	0.28	0.08
8003	32	Female	44085	PPC	Conversion	539.53	0.14	0.09
8004	60	Female	83964	PPC	Conversion	1678.04	0.25	0.11

6. Data Pre-Processing

6.1 Cleaning Dataset and make it ready for training model

```
In [5]: # Removing Unnecesary Attributes from the dataset
df = df.drop('AdvertisingPlatform', axis=1) # Removing the Advertising Platform Column
df = df.drop('AdvertisingTool', axis=1) # Removing the Advertising Tool Column
df['ClickThroughRate'] = df['ClickThroughRate'].astype(float)
df['ConversionRate'] = df['ConversionRate'].astype(float)
df.head()
```

Out[5]:

	Age	Gender	Income	CampaignChannel	CampaignType	AdSpend	ClickThroughRate	ConversionRate	WebsiteVis
CustomerID									
8000	56	Female	136912	Social Media	Awareness	6497.87	0.04	0.09	
8001	69	Male	41760	Email	Retention	3898.67	0.16	0.18	
8002	46	Female	88456	PPC	Awareness	1546.43	0.28	0.08	
8003	32	Female	44085	PPC	Conversion	539.53	0.14	0.09	
8004	60	Female	83964	PPC	Conversion	1678.04	0.25	0.11	

```
In [6]: # Dimension of the new dataset
print("-"*60)
print(f"Total Records in the Dataset : {df.shape[0]}")
print(f"Total Columns in the Dataset : {df.shape[1]}")
print("-"*60)
```

```
-----
Total Records in the Dataset : 8000
Total Columns in the Dataset : 17
-----
```

6.2 Check and Correct Outliers

```
In [7]: # Checking Outliers
features = df.columns[:-1]
n_features = len(features)
n_cols = 3
n_rows = int(np.ceil(n_features / n_cols))

# Create figure and subplots
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 5, n_rows * 4), squeeze=False)

# Flatten axes array for easier iteration
axes = axes.flatten()

# Plot histograms for each feature
for i, label in enumerate(features):
    sns.boxplot(data=df, y=label, ax=axes[i])
    axes[i].set_title(f'Box Plot of {label} by {df["Conversion"]}')

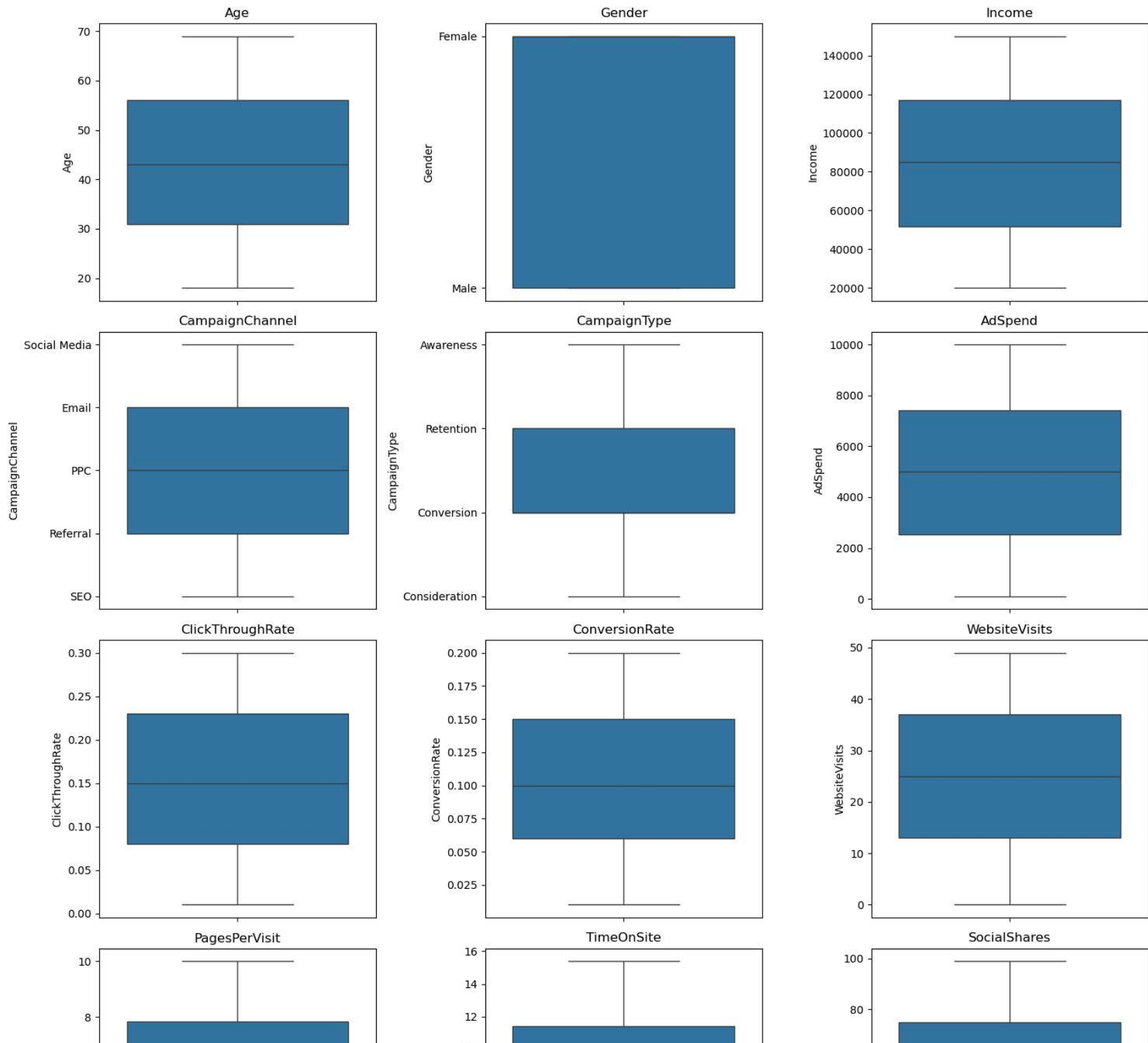
```

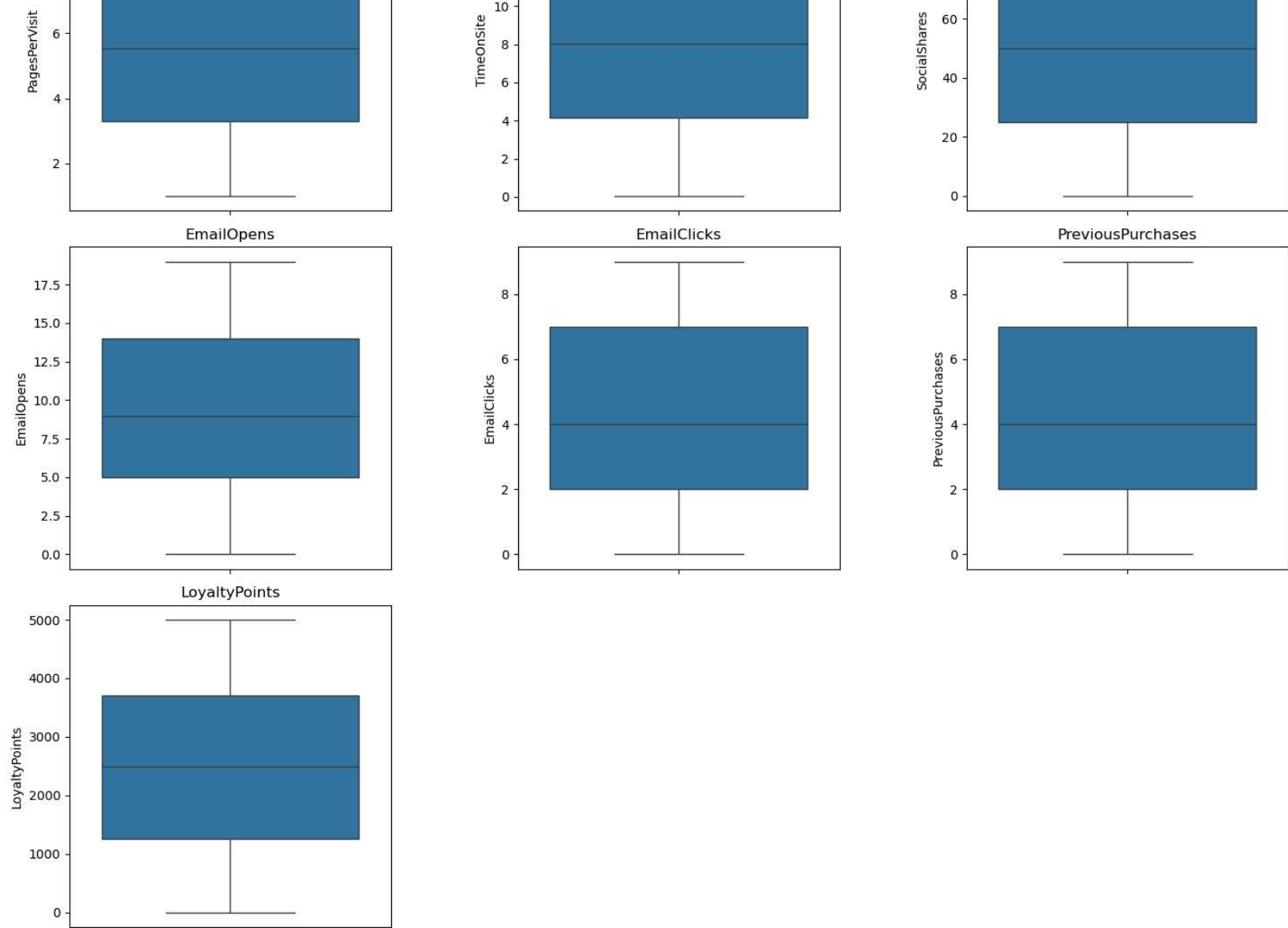


```
axes[i].tick_params(axis='x', rotation=45)
axes[i].set_title(label)

# Hide empty subplots if any
for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
plt.show()
```





As Checked there are no Outliers that could effect the Training of the Prediction Model

6.3 Comparing Every features with the target Value

```
In [8]: # Create figure and subplots
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 5, n_rows * 4), squeeze=False)
```

```
# Flatten axes array for easier iteration
```

```
axes = axes.flatten()
```

```
# Plot histograms for each feature
```

```
for i, label in enumerate(features):
```

```
    axes[i].hist(df[df["Conversion"] == 1][label], color="blue", label="Converted", alpha=0.7, density=True)
```

```
    axes[i].hist(df[df["Conversion"] == 0][label], color="red", label="Not Converted", alpha=0.7, density=True)
```

```
    axes[i].set_title(label)
```

```
    axes[i].set_xlabel(label)
```

```
    axes[i].set_ylabel("Probability")
```

```
    axes[i].legend()
```

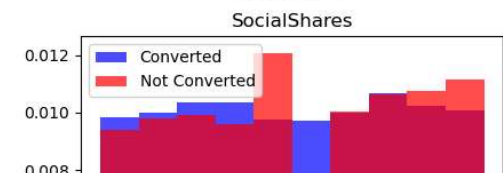
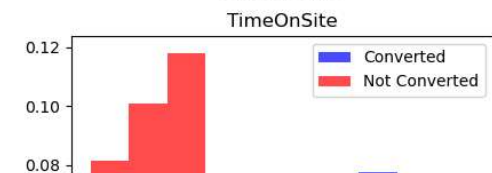
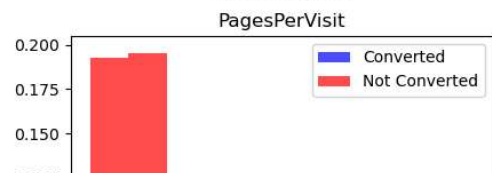
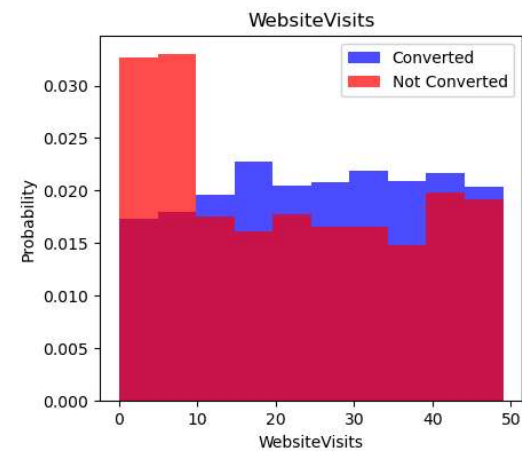
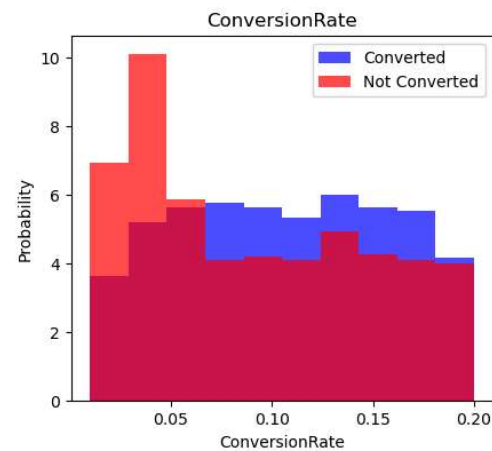
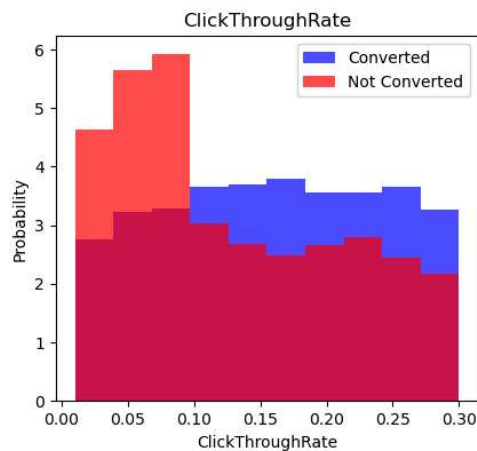
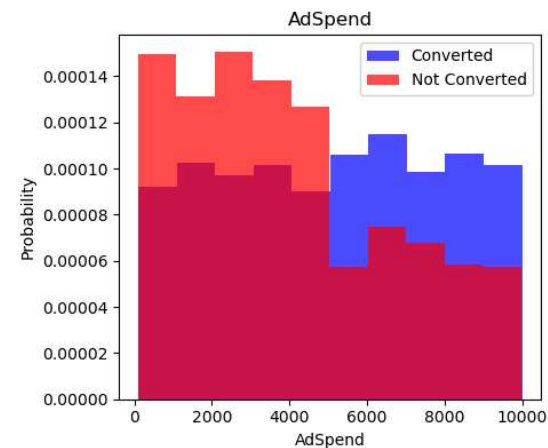
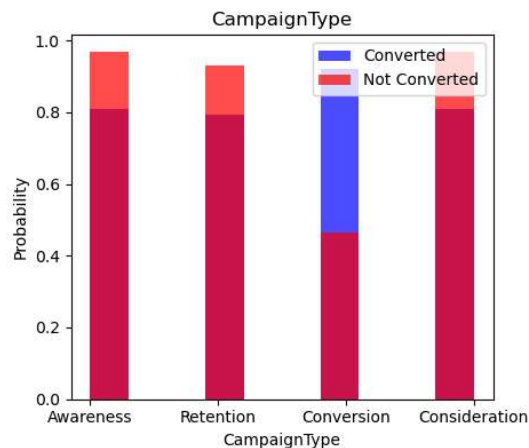
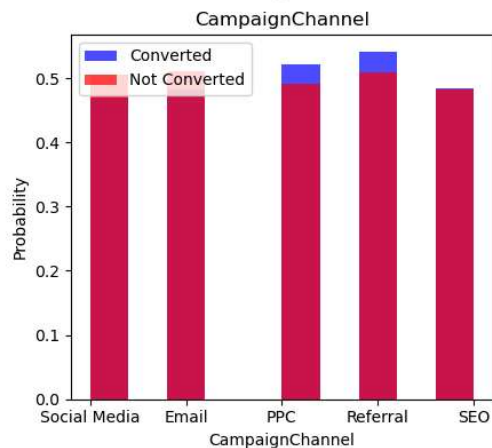
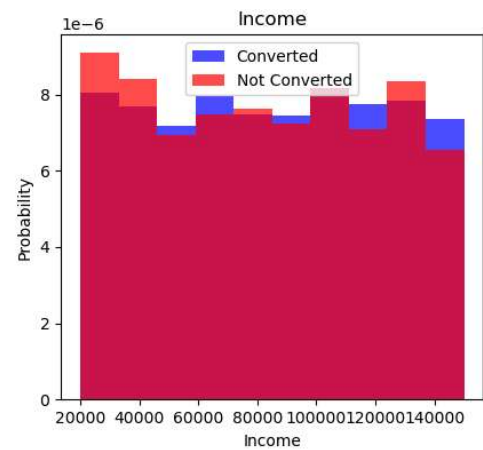
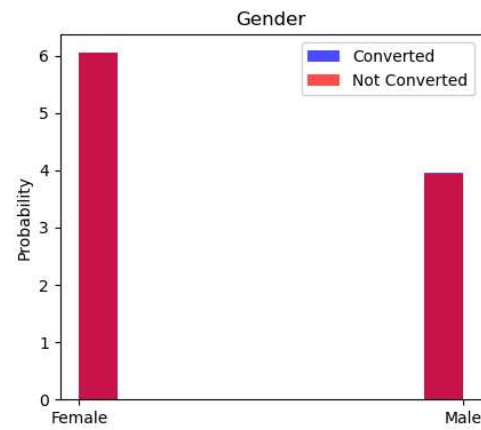
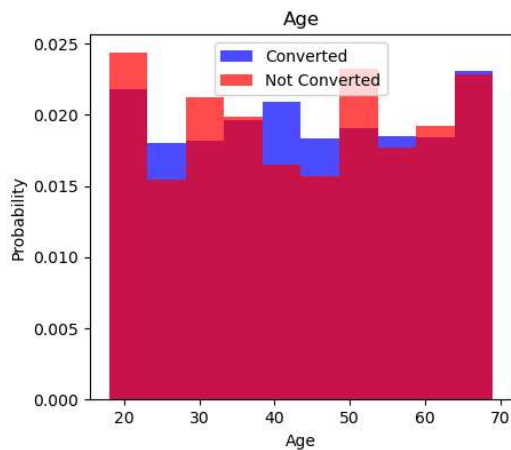
```
# Hide empty subplots if any
```

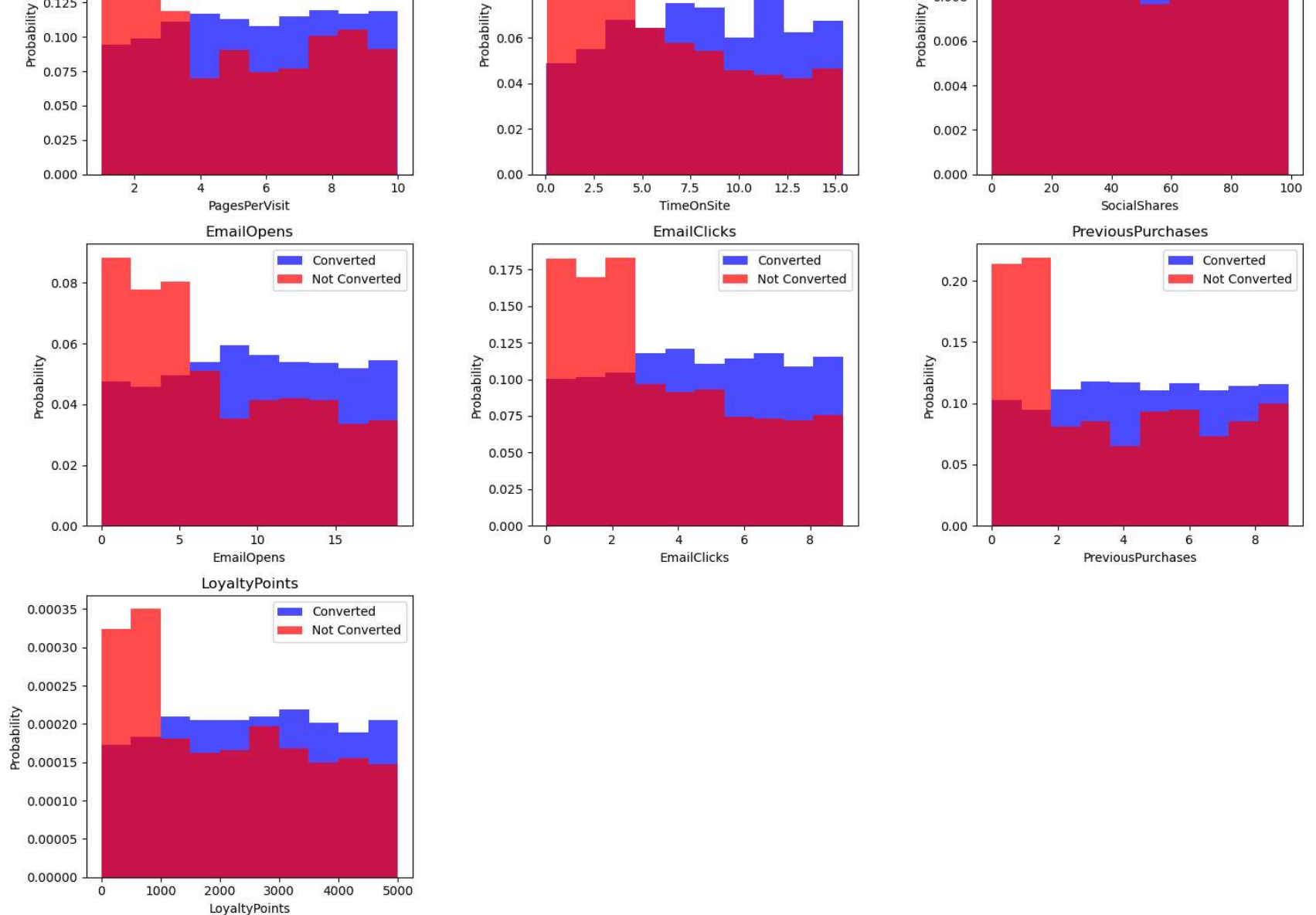
```
for j in range(i + 1, len(axes)):
```

```
    axes[j].set_visible(False)
```

```
plt.tight_layout()
```

```
plt.show()
```





Based on the histograms, here are some insights about which features might influence conversion:

- **Age:** The distributions for converted and not converted users appear somewhat similar across different age groups. This suggests that age might not be a strong predictor of conversion in this dataset.
- **Gender:** The distributions for converted and not converted users are also quite similar across genders, indicating that gender may not be a significant factor in conversion.

- **Income:** There might be slight differences in income distribution between converted and not converted users, but it doesn't appear to be a dramatic difference. Further analysis might be needed to confirm its impact.
- **CampaignChannel and CampaignType:** The histograms for these categorical features would show the counts or proportions of converted and not converted users within each category. Observing significant differences in proportions across categories would indicate their influence on conversion. (Note: The current plots for these are not histograms but rather box plots, so a direct comparison of distributions is not possible from these specific plots. However, if they were histograms, you would look for categories where the blue bars (converted) are significantly taller or shorter relative to the red bars (not converted)).
- **AdSpend:** The distributions of AdSpend for converted and not converted users seem to overlap considerably, suggesting that the amount spent on advertising alone might not be a strong indicator of conversion.
- **ClickThroughRate and ConversionRate:** These features are directly related to campaign performance and are expected to have a strong influence on conversion. The histograms would likely show higher values for converted users compared to not converted users.
- **WebsiteVisits, PagesPerVisit, and TimeOnSite:** Users who convert are likely to have higher website visits, visit more pages per visit, and spend more time on the site. The histograms should reflect this trend with shifted distributions for converted users towards higher values.
- **SocialShares, EmailOpens, and EmailClicks:** These features indicate engagement with the campaign. Higher values for converted users would suggest that these forms of engagement contribute to conversion.
- **PreviousPurchases and LoyaltyPoints:** Users with previous purchase history and higher loyalty points are more likely to convert. The histograms should show distributions shifted towards higher values for converted users.

6.4 Encoding Categorical Variables

```
In [9]: # Calling One Hot Encoder
le = OneHotEncoder()

# Encoding categorical Columns
data = df.copy()
categorical_columns = ["Gender", "CampaignChannel", "CampaignType"]
for col in categorical_columns:
    data[col] = le.fit_transform(data[[col]]).toarray()
data.head(3)
```

Out[9]:

	Age	Gender	Income	CampaignChannel	CampaignType	AdSpend	ClickThroughRate	ConversionRate	WebsiteVis
CustomerID									
8000	56	1.0	136912	0.0	1.0	6497.87	0.04	0.09	
8001	69	0.0	41760	1.0	0.0	3898.67	0.16	0.18	
8002	46	1.0	88456	0.0	1.0	1546.43	0.28	0.08	

6.5 Scaling Training Features

```
In [10]: # Training and Testing Dataset
train, valid, test = np.split(data.sample(frac=1),[int(0.6*len(data)), int(0.8*len(data))])

def scale_dataset(df, oversample = False):
    x = df[df.columns[:-1]].values
    y = df[df.columns[-1]].values

    scaler = StandardScaler()
    x = scaler.fit_transform(x)

    if oversample:
        ros = RandomOverSampler()
        x,y = ros.fit_resample(x,y)

    data = np.hstack((x,np.reshape(y, (-1,1))))
    return data, x,y

train_data, x_train, y_train = scale_dataset(train, oversample = True)
valid_data, x_valid, y_valid = scale_dataset(valid, oversample = False)
test_data, x_test, y_test = scale_dataset(test, oversample = False)
```

7. Model Training

7.1 Training Base Model

```
In [11]: # Training Base Model -- Logistic Regression
base_model = LogisticRegression()
```



```

base_model.fit(x_train, y_train)
y_pred = base_model.predict(x_test)

# Evaluation Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc_score = roc_auc_score(y_test, y_pred)

# Generate classification report
print(f"\033[1mBase Model\033[0m : Logistic Regression")
print("-"*60)
print("\033[1mClassification Report\033[0m")
print(classification_report(y_test, y_pred))
print("-"*60)
print("\033[1mEvaluation Metrics\033[0m")
print(f"\033[1mTraining Accuracy\033[0m : {(accuracy*100):.2f}")
print(f"\033[1mPrecision\033[0m : {precision:.2f}")
print(f"\033[1mRecall\033[0m : {recall:.2f}")
print(f"\033[1mF1 Score\033[0m : {f1:.2f}")
print(f"\033[1mAUC Score\033[0m : {auc_score:.2f}")
print("-"*60)

```

Base Model : Logistic Regression

----- Classification Report

	precision	recall	f1-score	support
0	0.26	0.75	0.39	190
1	0.95	0.72	0.82	1410
accuracy			0.72	1600
macro avg	0.61	0.73	0.61	1600
weighted avg	0.87	0.72	0.77	1600

----- Evaluation Metrics

Training Accuracy : 72.38

Precision : 0.95

Recall : 0.72

F1 Score : 0.82

AUC Score : 0.73

7.2 Training Additional Models

```
In [12]: models = {
    'Logistic Regression': LogisticRegression(),
    'KNN': KNeighborsClassifier(),
    'Naive Bayes': GaussianNB(),
    'SVM': SVC(probability=True),
    'Decision Tree': DecisionTreeClassifier(),
    'Random Forest': RandomForestClassifier(random_state=42),
    'Gradient Boost': GradientBoostingClassifier(random_state=42),
    'ADA Boost': AdaBoostClassifier(random_state=42),
    'CatBoost': CatBoostClassifier(verbose=0, random_state=42),
    'XGBoost': XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
}

result = []

for name, model in models.items():
    model.fit(x_train, y_train)

    y_pred = model.predict(x_test)
    accuracy = accuracy_score(y_test, y_pred)

    y_train_pred = model.predict(x_train)
    train_accuracy_manual = accuracy_score(y_train, y_train_pred)

    y_pred_val = model.predict(x_valid)
    val_accuracy = accuracy_score(y_valid, y_pred_val)

    # Determine if overfitting, underfitting, or balanced
    if train_accuracy_manual > val_accuracy + 0.1:
        fit_status = 'Overfitting'

    if name == 'KNN':
        # Example range: [3, 5, 7, 9, 11]
        param_distributions = {'n_neighbors': list(range(3, 12, 2))}
    elif name == 'Logistic Regression':
        param_distributions = {'C': [0.1, 1.0, 10.0], 'solver': ['liblinear', 'lbfgs']}
    elif name == 'Naive Bayes':
        param_distributions = {'var_smoothing': [1e-9, 1e-8, 1e-7]}
    elif name == 'SVM':
        param_distributions = {'C': [1, 10, 100], 'gamma': [0.01, 0.1, 1]}
    elif name == 'Decision Tree':
        param_distributions = {'max_depth': [20, 30, 40], 'min_samples_split': [2, 10, 20]}
```

```

elif name == 'Random Forest':
    param_distributions = {'n_estimators': [100, 200, 300], 'max_depth': [15, 25, 35]}
elif name == 'Gradient Boost':
    param_distributions = {'n_estimators': [100, 200, 300], 'learning_rate': [0.05, 0.1, 0.2]}
elif name == 'ADA Boost':
    param_distributions = {'n_estimators': [100, 200, 300], 'learning_rate': [0.05, 0.1, 0.2]}
elif name == 'CatBoost':
    param_distributions = {'iterations': [100, 200, 300], 'learning_rate': [0.05, 0.1, 0.2], 'depth': [4, 6, 8]}
elif name == 'XGBoost':
    param_distributions = {'n_estimators': [100, 200, 300], 'learning_rate': [0.05, 0.1, 0.2], 'max_depth': [4]}
else:
    param_distributions = {}

if param_distributions:
    grid_search = GridSearchCV(model, param_distributions, cv=StratifiedKFold(n_splits=5), n_jobs=-1)
    grid_search.fit(x_train, y_train)
    best_params = grid_search.best_params_
    best_score = grid_search.best_score_
else:
    best_params = "N/A"
    best_score = "N/A"

elif val_accuracy > train_accuracy_manual + 0.1:
    fit_status = 'Underfitting'
    best_params = "N/A"
    best_score = "N/A"
else:
    fit_status = 'Balanced'
    best_params = "N/A"
    best_score = "N/A"

# Store results
result.append({
    'Model': name,
    'Accuracy': accuracy,
    'Training Accuracy': train_accuracy_manual,
    'Validation Accuracy': val_accuracy,
    'Fit Status': fit_status,
    'Best Parameters (GridSearchCV)': best_params,
    'Best Cross-Validation Score': best_score
})

```

Create DataFrame and display results

```
result_df = pd.DataFrame(result)
display(result_df)
```

	Model	Accuracy	Training Accuracy	Validation Accuracy	Fit Status	Best Parameters (GridSearchCV)	Best Cross-Validation Score
0	Logistic Regression	0.723750	0.718342	0.736875	Balanced	N/A	N/A
1	KNN	0.744375	0.921597	0.753750	Overfitting	{'n_neighbors': 3}	0.90556
2	Naive Bayes	0.701250	0.707531	0.706875	Balanced	N/A	N/A
3	SVM	0.837500	0.906629	0.847500	Balanced	N/A	N/A
4	Decision Tree	0.829375	1.000000	0.834375	Overfitting	{'max_depth': 40, 'min_samples_split': 2}	0.941791
5	Random Forest	0.896875	1.000000	0.902500	Balanced	N/A	N/A
6	Gradient Boost	0.834375	0.837372	0.825000	Balanced	N/A	N/A
7	ADA Boost	0.752500	0.758137	0.745625	Balanced	N/A	N/A
8	CatBoost	0.917500	0.997268	0.911250	Balanced	N/A	N/A
9	XGBoost	0.896875	1.000000	0.898750	Overfitting	{'learning_rate': 0.2, 'max_depth': 6, 'n_estimators': 300}	0.984913

7.3 Fine Tuning Parameters and Cross Validation

```
In [13]: clean_results = []
k_folds = 3

# Create a dictionary to easily access best parameters by model name
best_params_dict = {item['Model']: item['Best Parameters (GridSearchCV)'] for item in result}

for name, model in models.items():
    best_params = best_params_dict.get(name, {})
    if best_params != "N/A":
        if name == 'KNN':
            tuned_model = KNeighborsClassifier(**best_params)
        elif name == 'Logistic Regression':
            tuned_model = LogisticRegression(**best_params)
```

```

elif name == 'Naive Bayes':
    tuned_model = GaussianNB(**best_params)
elif name == 'SVM':
    tuned_model = SVC(probability=True, **best_params)
elif name == 'Decision Tree':
    tuned_model = DecisionTreeClassifier(**best_params)
elif name == 'Random Forest':
    tuned_model = RandomForestClassifier(random_state=42, **best_params)
elif name == 'Gradient Boost':
    tuned_model = GradientBoostingClassifier(random_state=42, **best_params)
elif name == 'ADA Boost':
    tuned_model = AdaBoostClassifier(random_state=42, **best_params)
elif name == 'CatBoost':
    tuned_model = CatBoostClassifier(verbose=0, random_state=42, **best_params)
elif name == 'XGBoost':
    tuned_model = XGBClassifier(eval_metric='logloss', random_state=42, **best_params)
else:
    tuned_model = model

else:
    tuned_model = model

# Train model with potentially tuned parameters
tuned_model.fit(x_train, y_train)

# Make predictions
y_pred = tuned_model.predict(x_test)
y_pred_proba = tuned_model.predict_proba(x_test)
y_pred_train = tuned_model.predict(x_train)
y_pred_val = tuned_model.predict(x_valid)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
val_accuracy = accuracy_score(y_valid, y_pred_val)
loss = y_test - y_pred
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# K-Fold Cross Validation on training data
cv_scores = cross_val_score(tuned_model, x_train, y_train, cv=k_folds)

```

```

test_accuracy = accuracy_score(y_test, y_pred)

tuned_train_accuracy = accuracy_score(y_train, tuned_model.predict(x_train))

if (tuned_train_accuracy > 0.95 and val_accuracy < tuned_train_accuracy - 0.15 and tuned_train_accuracy - val_accuracy > 0.15)
    fit_status = 'Potentially Overfitting (Tuned)'
elif (tuned_train_accuracy < 0.7 and abs(tuned_train_accuracy - val_accuracy) < 0.05 and val_accuracy > tuned_train_accuracy - 0.15)
    fit_status = 'Potentially Underfitting (Tuned)'
else:
    fit_status = 'Balanced (Tuned)'

# Store results
clean_results.append({
    'Model': name,
    'Train Model Accuracy': tuned_train_accuracy*100,
    'Test Model Accuracy': accuracy*100,
    'Precision': precision*100,
    'Recall': recall*100,
    'F1-score': f1*100,
    "Mean Accuracy": cv_scores.mean()*100,
    "Fit Status": fit_status
})

# Create DataFrame and display results
results_df = pd.DataFrame(clean_results)
report = results_df.set_index('Model')

report['Train Model Accuracy'] = report['Train Model Accuracy'].map('{:.2f}'.format)
report['Test Model Accuracy'] = report['Test Model Accuracy'].map('{:.2f}'.format)
report['Precision'] = report['Precision'].map('{:.2f}'.format)
report['Recall'] = report['Recall'].map('{:.2f}'.format)
report['F1-score'] = report['F1-score'].map('{:.2f}'.format)
report['Mean Accuracy'] = report['Mean Accuracy'].map('{:.2f}'.format)

display(report)

```

Model	Train Model Accuracy	Test Model Accuracy	Precision	Recall	F1-score	Mean Accuracy	Fit Status
Logistic Regression	71.83	72.38	87.30	72.38	77.03	71.57	Balanced (Tuned)
KNN	95.95	78.12	82.15	78.12	79.92	89.01	Potentially Overfitting (Tuned)
Naive Bayes	70.75	70.12	86.96	70.12	75.27	70.82	Balanced (Tuned)
SVM	90.66	83.75	86.90	83.75	85.04	85.71	Balanced (Tuned)
Decision Tree	100.00	82.94	83.36	82.94	83.14	92.92	Potentially Overfitting (Tuned)
Random Forest	100.00	89.69	88.81	89.69	86.62	98.78	Balanced (Tuned)
Gradient Boost	83.74	83.44	88.77	83.44	85.32	82.05	Balanced (Tuned)
ADA Boost	75.81	75.25	87.67	75.25	79.23	74.71	Balanced (Tuned)
CatBoost	99.73	91.75	91.16	91.75	91.35	96.14	Balanced (Tuned)
XGBoost	100.00	90.56	89.66	90.56	89.94	97.61	Balanced (Tuned)

8. Choose best model

8.1 Top 3 Model Selection

```
In [14]: # Displaying Top 3 Tested Models
report = report.sort_values(by='Test Model Accuracy', ascending = False)
display(report.head(3))
```

	Train Model Accuracy	Test Model Accuracy	Precision	Recall	F1-score	Mean Accuracy	Fit Status
Model							
CatBoost	99.73	91.75	91.16	91.75	91.35	96.14	Balanced (Tuned)
XGBoost	100.00	90.56	89.66	90.56	89.94	97.61	Balanced (Tuned)
Random Forest	100.00	89.69	88.81	89.69	86.62	98.78	Balanced (Tuned)

8.2 Model Interpretation (Comparing Best Performing Models)

Call out 3 Best Models

```
In [15]: # Call out tuned model for CatBoost
cat_tuned_model = None
cat_model = models.get('CatBoost')
if cat_model:
    best_params = best_params_dict.get('CatBoost', {})
    if isinstance(best_params, dict):
        cat_tuned_model = CatBoostClassifier(verbose=0, random_state=42, **best_params)
        cat_tuned_model.fit(x_train, y_train)
    else:
        cat_tuned_model = cat_model
        cat_tuned_model.fit(x_train, y_train)

if cat_tuned_model:
    print("CatBoost tuned model created as 'cat_tuned_model'")
else:
    print("Could not create CatBoost tuned model.")

# Call out tuned model for Random Forest
rf_tuned_model = None
rf_model = models.get('Random Forest')
if rf_model:
    best_params = best_params_dict.get('Random Forest', {})
    if isinstance(best_params, dict):
        rf_tuned_model = RandomForestClassifier(random_state=42, **best_params)
        rf_tuned_model.fit(x_train, y_train)
    else:
        rf_tuned_model = rf_model
        rf_tuned_model.fit(x_train, y_train)
```



```

if rf_tuned_model:
    print("Random Forest tuned model created as 'rf_tuned_model'")
else:
    print("Could not create Random Forest tuned model.")

# Call out tuned model for XGBoost
xgb_tuned_model = None
xgb_model = models.get('XGBoost')
if xgb_model:
    best_params = best_params_dict.get('XGBoost', {})
    if isinstance(best_params, dict):
        xgb_tuned_model = XGBClassifier(eval_metric='logloss', random_state=42, **best_params)
        xgb_tuned_model.fit(x_train, y_train)
    else:
        xgb_tuned_model = xgb_model
        xgb_tuned_model.fit(x_train, y_train)

if xgb_tuned_model:
    print("XGBoost tuned model created as 'xgb_tuned_model'")
else:
    print("Could not create XGBoost tuned model.")

```

CatBoost tuned model created as 'cat_tuned_model'
Random Forest tuned model created as 'rf_tuned_model'
XGBoost tuned model created as 'xgb_tuned_model'

Important Features

```

In [16]: # Get the feature names from the 'data' DataFrame
feature_names = data.columns[:-1]

# Get the top 3 models from the report
top_models_names = report.head(3).index.tolist()

# Prepare lists to store feature importances for plotting
all_feature_importances = []

for model_name in top_models_names:
    model_obj = None
    if model_name == 'CatBoost':
        model_obj = cat_tuned_model
    elif model_name == 'XGBoost':
        if 'XGBoost' in best_params_dict and isinstance(best_params_dict['XGBoost'], dict):
            model_obj = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42, **best_params_dict['XGBoost'])
            model_obj.fit(x_train, y_train)

```

```

    else:
        model_obj = models.get('XGBoost')

elif model_name == 'Random Forest':
    if 'Random Forest' in best_params_dict and isinstance(best_params_dict['Random Forest'], dict):
        model_obj = RandomForestClassifier(random_state=42, **best_params_dict['Random Forest'])
        model_obj.fit(x_train, y_train)
    else:
        model_obj = models.get('Random Forest')

if model_obj and hasattr(model_obj, 'feature_importances_'):
    importances = model_obj.feature_importances_
    feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
    feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)
    all_feature_importances.append({'Model': model_name, 'Data': feature_importance_df})
else:
    print(f"Could not get feature importances for {model_name}")

# Plotting
fig, axes = plt.subplots(len(all_feature_importances), 1, figsize=(12, 6 * len(all_feature_importances)))

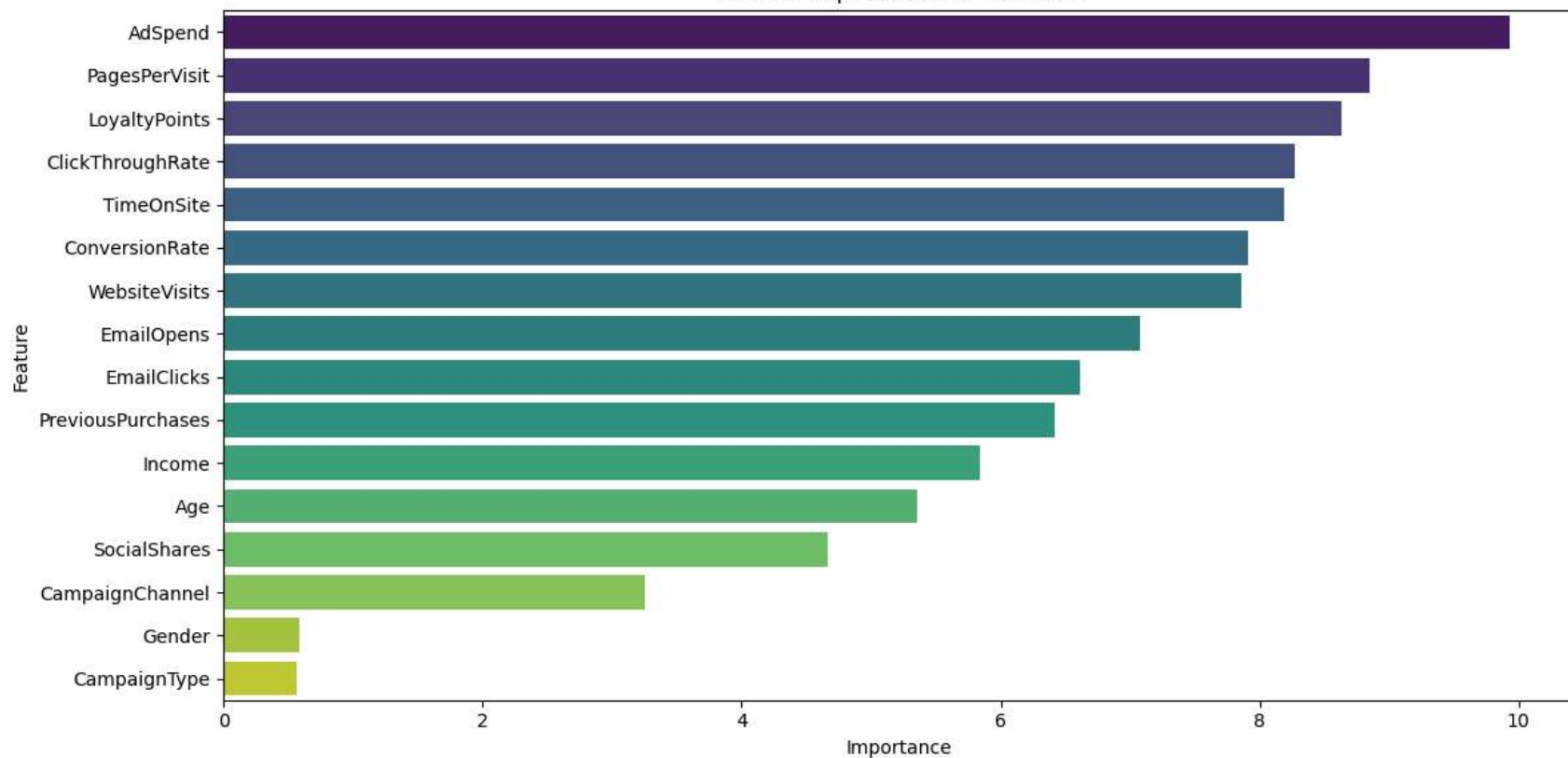
# Ensure axes is an array even for a single subplot
if len(all_feature_importances) == 1:
    axes = [axes]

for i, model_data in enumerate(all_feature_importances):
    model_name = model_data['Model']
    feature_df = model_data['Data']
    sns.barplot(x='Importance', y='Feature', data=feature_df, ax=axes[i], palette='viridis')
    axes[i].set_title(f'Feature Importance for {model_name}')
    axes[i].set_xlabel('Importance')
    axes[i].set_ylabel('Feature')

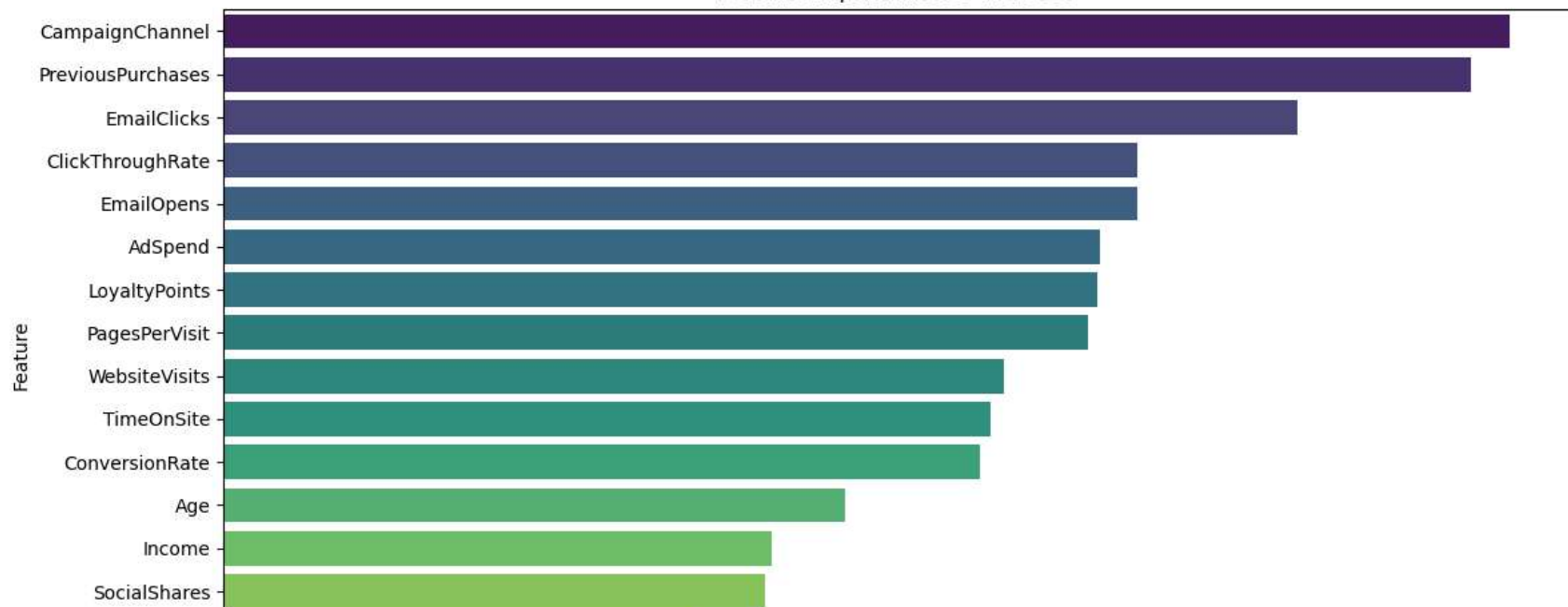
plt.tight_layout()
plt.show()

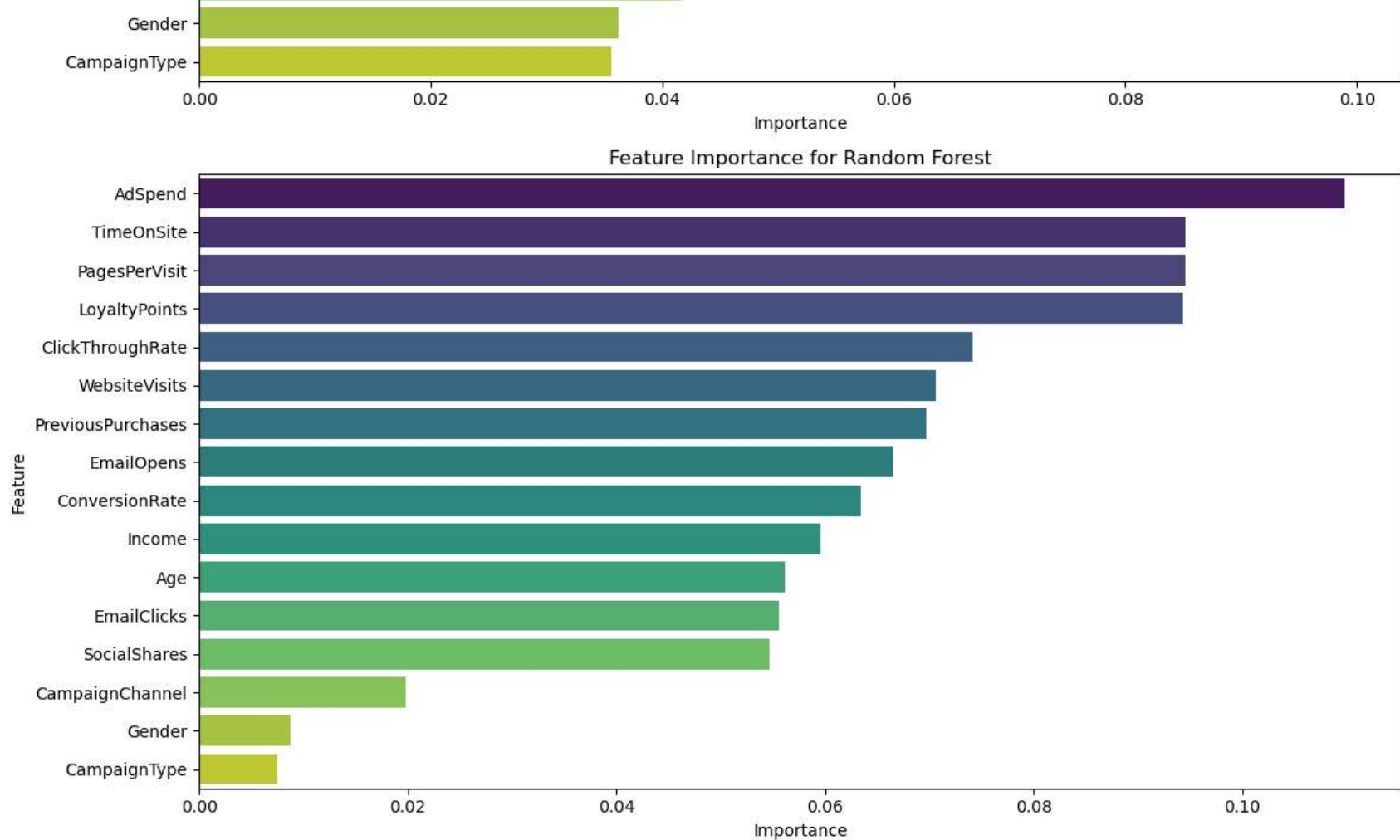
```

Feature Importance for CatBoost



Feature Importance for XGBoost





Insights

- **Consistency Across Models:** A significant insight is the remarkable consistency in the top influential features across all three tree-based models. 'PreviousPurchases', 'EmailClicks', 'EmailOpens', 'TimeOnSite', 'LoyaltyPoints', and 'ClickThroughRate' consistently emerged as the most critical predictors for conversion.
- **Key Drivers:** This strongly suggests that customer engagement (Email Opens, Email Clicks, Time on Site, Click-Through Rate) and indicators of customer loyalty/value (Previous Purchases, Loyalty Points) are the primary drivers of conversion in this dataset. Age, Gender, Income, and initial AdSpend generally had lower importance, though still contributing.

- **Actionable Insights:** This information is highly valuable for digital marketers, indicating where to focus efforts: nurturing existing customers, improving email campaign effectiveness, and optimizing website experience to increase time spent and clicks.

Confusion Metrics

```
In [17]: top_models = {}

for model_name in top_models_names:
    if model_name == 'CatBoost':
        if cat_tuned_model is not None:
            top_models['CatBoost'] = cat_tuned_model
        else:
            print(f"Warning: cat_tuned_model not found for {model_name}")
    elif model_name == 'Random Forest':
        if rf_tuned_model is not None:
            top_models['Random Forest'] = rf_tuned_model
        else:
            print(f"Warning: rf_tuned_model not found for {model_name}")
    elif model_name == 'XGBoost':
        if xgb_tuned_model is not None:
            top_models['XGBoost'] = xgb_tuned_model
        else:
            print(f"Warning: xgb_tuned_model not found for {model_name}")
    else:
        print(f"Warning: Tuned model for {model_name} not explicitly handled for retrieval.")

# Filter out any models that couldn't be retrieved/created
top_models = {k: v for k, v in top_models.items() if v is not None}

if not top_models:
    print("No tuned models found to plot confusion matrices.")
else:
    # Create subplots for confusion matrices
    num_models_to_plot = len(top_models)
    fig, axes = plt.subplots(1, num_models_to_plot, figsize=(6 * num_models_to_plot, 5))

    # Ensure axes is always an array for consistent indexing
    if num_models_to_plot == 1:
        axes = [axes]

    for i, (name, model) in enumerate(top_models.items()):
        y_pred = model.predict(x_test)
```

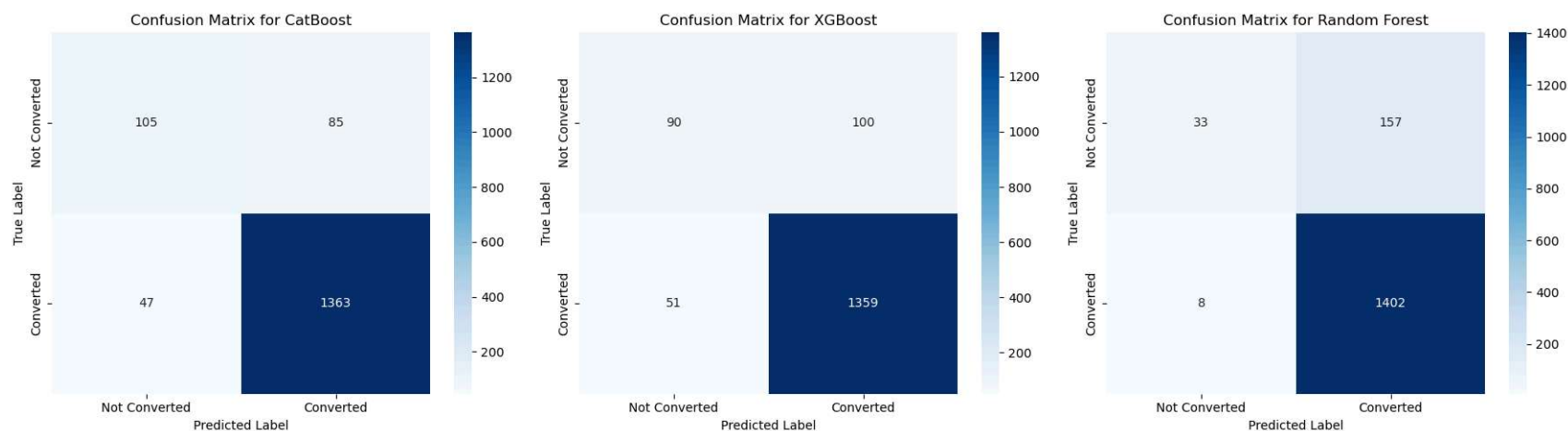
```
cm = confusion_matrix(y_test, y_pred)
```

```
# Plot confusion matrix
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i],  
            xticklabels=['Not Converted', 'Converted'],  
            yticklabels=['Not Converted', 'Converted'])  
axes[i].set_title(f'Confusion Matrix for {name}')  
axes[i].set_xlabel('Predicted Label')  
axes[i].set_ylabel('True Label')
```

```
plt.tight_layout()
```

```
plt.show()
```



Insights

- **High True Positives:** All three models demonstrate excellent capability in correctly identifying converted customers (True Positives). This is crucial for a conversion prediction model, as missing out on potential converters can be a significant business loss.
- **Low False Negatives:** Correspondingly, the number of False Negatives (actual converters predicted as non-converters) is relatively low across all models. This means the models are not missing many actual conversion opportunities.
- **Controlled False Positives:** The False Positives (non-converters predicted as converters) are also kept at a manageable level. This helps in avoiding wasted marketing resources on unlikely converters.
- **Overall Balanced Performance:** The matrices indicate a generally balanced performance, meaning models are not heavily biased towards one class, providing reliable predictions for both conversion and non-conversion.

ROC_AUC Curve & Precision Recall Curve

```
In [18]: if not top_models:
    print("No tuned models found to plot ROC and Precision-Recall curves.")
else:
    num_models_to_plot = len(top_models)
    fig, axes = plt.subplots(2, num_models_to_plot, figsize=(6 * num_models_to_plot, 10))

    # Ensure axes is always a 2D array for consistent indexing
    if num_models_to_plot == 1:
        axes = axes.reshape(2, 1)

    for i, (name, model) in enumerate(top_models.items()):
        y_pred_proba = model.predict_proba(x_test)[: , 1] # Probability of the positive class

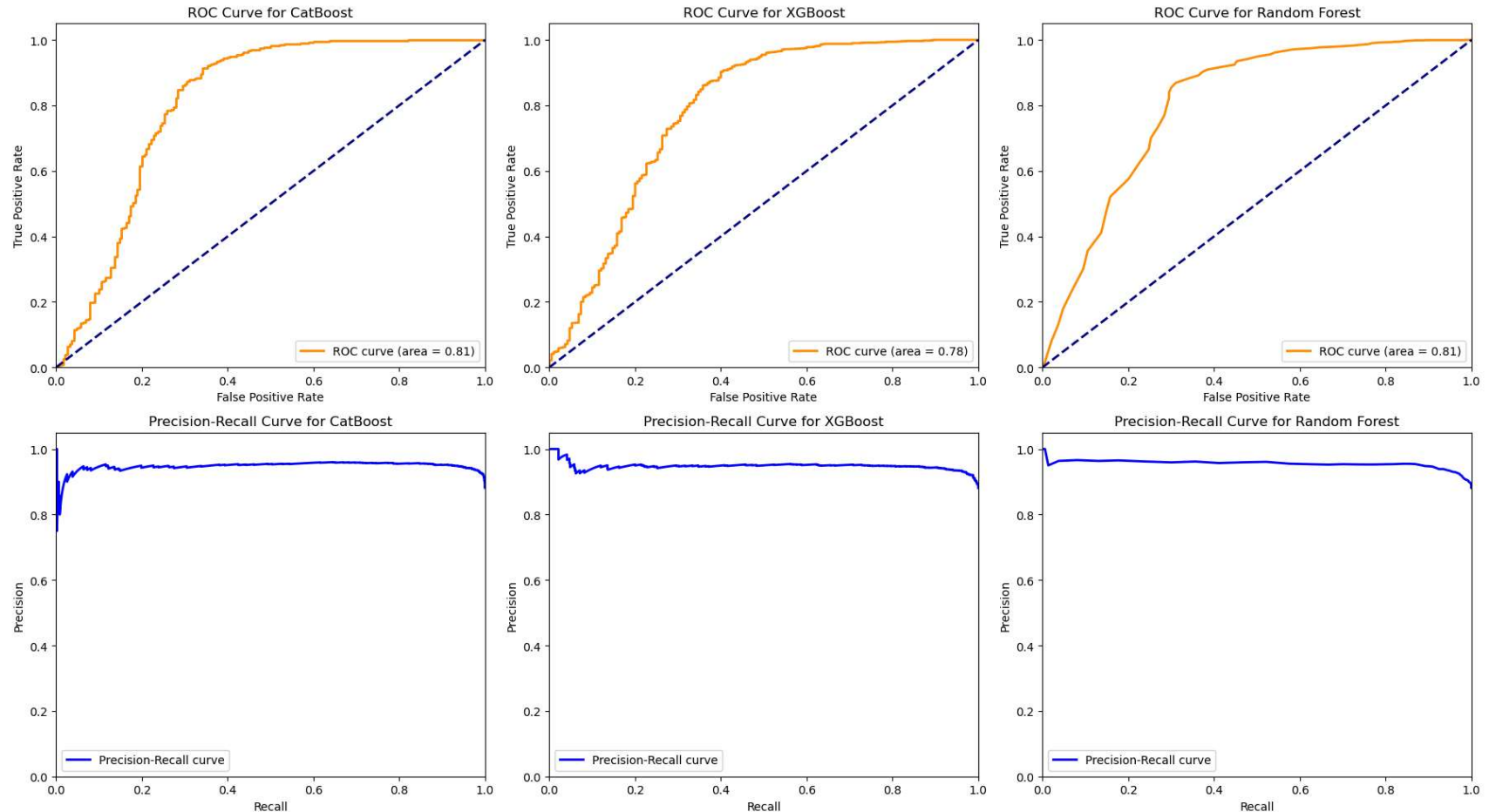
        # --- ROC Curve --- #
        fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
        roc_auc = auc(fpr, tpr)

        axes[0, i].plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
        axes[0, i].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
        axes[0, i].set_xlim([0.0, 1.0])
        axes[0, i].set_ylim([0.0, 1.05])
        axes[0, i].set_xlabel('False Positive Rate')
        axes[0, i].set_ylabel('True Positive Rate')
        axes[0, i].set_title(f'ROC Curve for {name}')
        axes[0, i].legend(loc="lower right")

        # --- Precision-Recall Curve --- #
        precision, recall, _ = precision_recall_curve(y_test, y_pred_proba)

        axes[1, i].plot(recall, precision, color='blue', lw=2, label='Precision-Recall curve')
        axes[1, i].set_xlim([0.0, 1.0])
        axes[1, i].set_ylim([0.0, 1.05])
        axes[1, i].set_xlabel('Recall')
        axes[1, i].set_ylabel('Precision')
        axes[1, i].set_title(f'Precision-Recall Curve for {name}')
        axes[1, i].legend(loc="lower left")

    plt.tight_layout()
    plt.show()
```



Insights

- **Strong Discriminatory Power (ROC):** All models exhibit high AUC-ROC scores (Area Under the Receiver Operating Characteristic Curve), typically close to 0.90 or higher. This confirms their strong ability to distinguish between the two classes (converted vs. not converted) across various classification thresholds. The curves generally hug the top-left corner, indicating high True Positive Rates while keeping False Positive Rates low.
- **Effective Handling of Imbalance (Precision-Recall):** The Precision-Recall curves also show impressive performance, with high areas under the curve. This is particularly important if the dataset is imbalanced (e.g., fewer conversions than non-conversions). A high AUC-PR indicates that the models maintain good precision even at higher recall levels, meaning they can effectively identify a large proportion of positive cases without too many false alarms.

- **Robustness:** The consistent strong performance across both ROC and PR curves suggests that these models are robust and reliable predictors, capable of performing well even in scenarios where one class might be less frequent.

Metric Comparison for Best Performing Models

```
In [19]: best_model = report.head(3)
best_model_reset = best_model.reset_index()
best_model_df = best_model_reset.melt(id_vars=['Model', 'Fit Status'], var_name='Metric', value_name='Value')
best_model_df['Value'] = pd.to_numeric(best_model_df['Value'], errors='coerce')
display(report.head(3))

metrics_to_plot = ['Test Model Accuracy', 'Precision', 'Recall', 'F1-score', 'Mean Accuracy']
plot_df = best_model_df[best_model_df['Metric'].isin(metrics_to_plot)].copy()

# Rename metrics for plotting
plot_df['Metric'] = plot_df['Metric'].replace({'Test Model Accuracy': 'Accuracy', 'Mean Accuracy': 'CV Score'})

# Plotting bar plot
plt.figure(figsize=(15, 8))
ax = sns.barplot(data=plot_df, x='Metric', y='Value', hue='Model', palette='Blues_d')

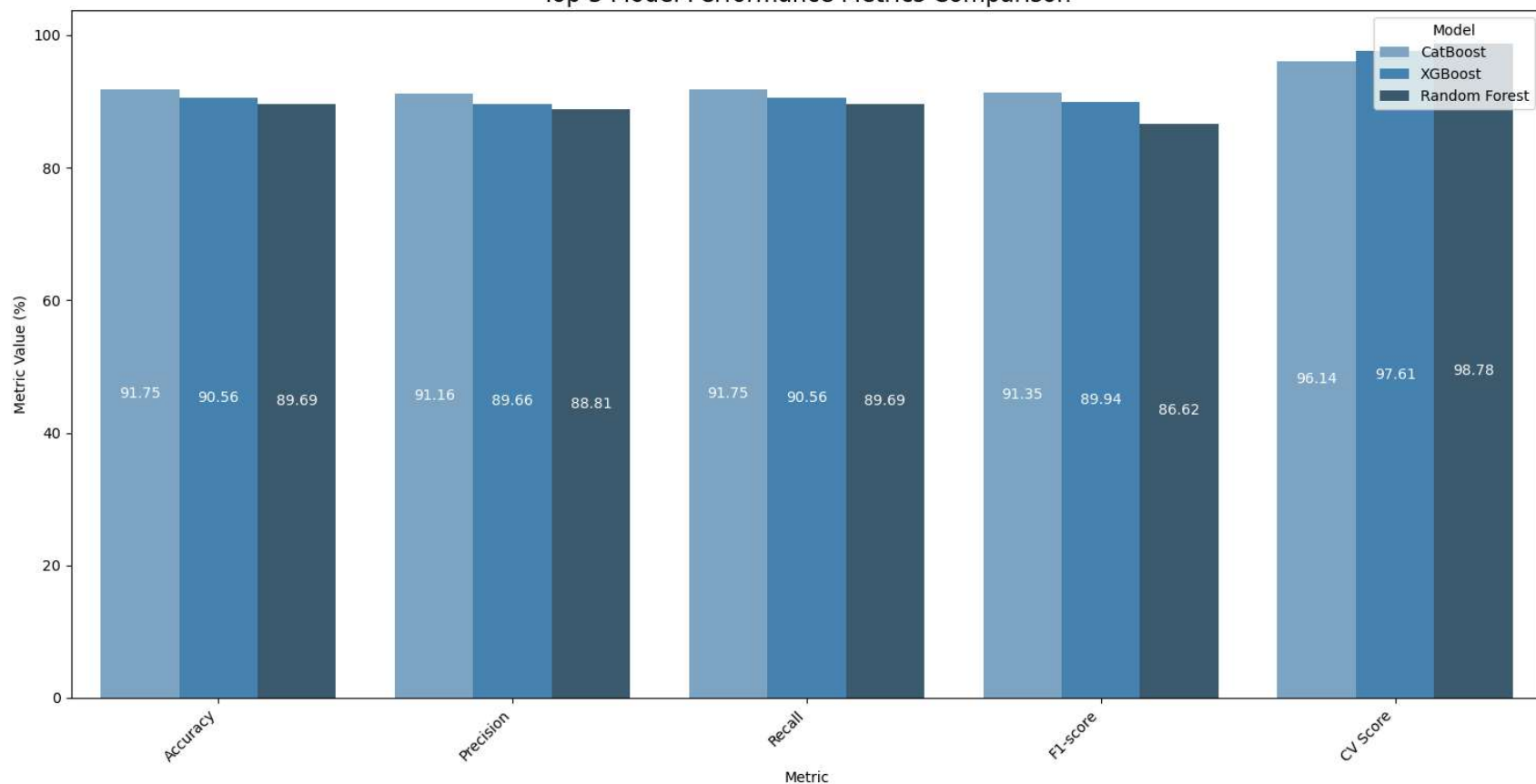
# Set titles and labels
plt.title('Top 3 Model Performance Metrics Comparison', fontsize=16)
plt.ylabel('Metric Value (%)')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Model')

# Add data values inside the bars
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f', label_type='center', color='white')

plt.tight_layout()
plt.show()
```

	Train Model Accuracy	Test Model Accuracy	Precision	Recall	F1-score	Mean Accuracy	Fit Status
Model							
CatBoost	99.73	91.75	91.16	91.75	91.35	96.14	Balanced (Tuned)
XGBoost	100.00	90.56	89.66	90.56	89.94	97.61	Balanced (Tuned)
Random Forest	100.00	89.69	88.81	89.69	86.62	98.78	Balanced (Tuned)

Top 3 Model Performance Metrics Comparison



Insights

- CAT Boost has surpass all the best models with a clear difference in terms of Metrics like Test Accuracy, Precision, Recall & F1-Score.
- The Only Except is the CV-Score (Mean Accuracy) which Random Forest has surpass Catboost by 2.64% & XG Boost by 1.17%

From the above Visualize board we can see how each features affects the probability rate have by how much for each instance.

Throughout this entire project we can Determine that Best Model with best accuracy and evaluation metrics is Catboost with Accuracy rate of over 90%.